جامعة الملك عبدالله للعلوم والتقنية
King Abdullah University of Science and Technology

أكاديمية كاوست
KAUST ACADEMY

# Internet of Things (IoT)

**DAY 2** From analog signals to a board your phone can control

August 2 to 6, 2026

KGSP Summer Enrichment Program 2026
KAUST Academy, KAUST

Salah Abdeljabar
salah.abdeljabar@kaust.edu.sa

# 🗺️ On Today's Agenda

1. **Analog vs digital sensors**: the ADC, and sampling a continuous world
2. 🔧 **Project 2**: read a potentiometer as a number from 0 to 4095
3. **Controlling actuators**: the DAC, and faking analog with PWM
4. 🔧 **Project 3**: fade an LED smoothly instead of switching it
5. **Anatomy of an IoT application**: the thing, the internet, the decision
6. **How things talk**: publish/subscribe, HTTP, and the protocol landscape
7. 🔧 **Project 5**: control your board from a web page on your phone

By the end your board will be on the network, taking orders from your phone.

# Analog vs Digital Sensors 

...and how a digital device reads an analog world

# Analog Sensors

An **analog** sensor returns a **variable voltage** that maps to what it measures.

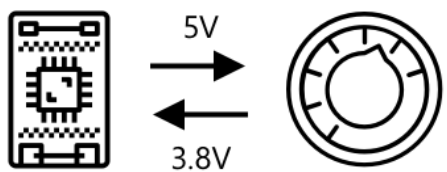
### 🎚️ Potentiometer

5 V in → **0–5 V** out as the dial turns

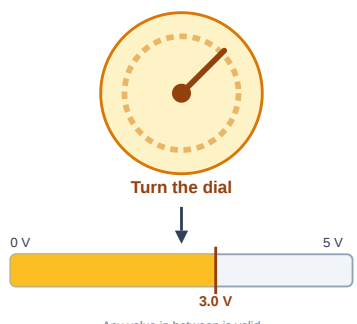
### 🌡️ Thermistor

voltage in → **°C** out, converted in code

The value can be anything in that range, not just fixed steps.



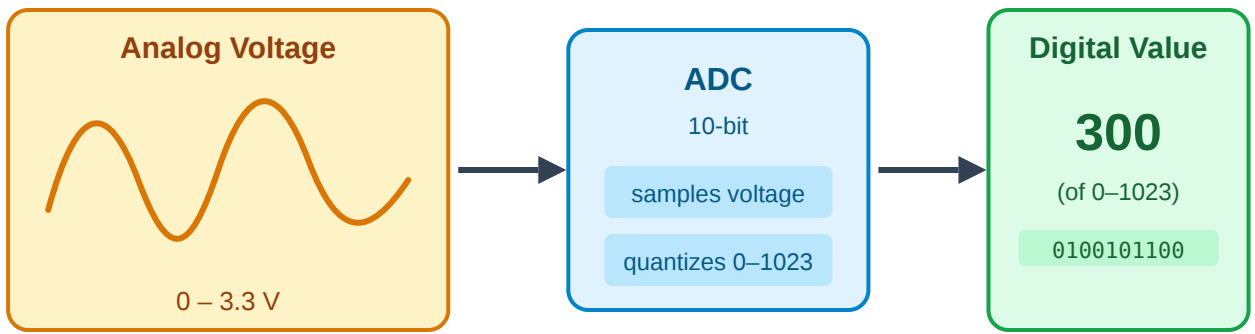

Turning the dial changes the voltage the potentiometer sends back, anywhere between 0 V and 5 V.




The output can land on any point along the range, not just a fixed set of steps.

# The ADC: Analog → Digital

IoT devices are **digital**: they only understand 0s and 1s. An **analog-to-digital converter (ADC)** turns a voltage into a number.

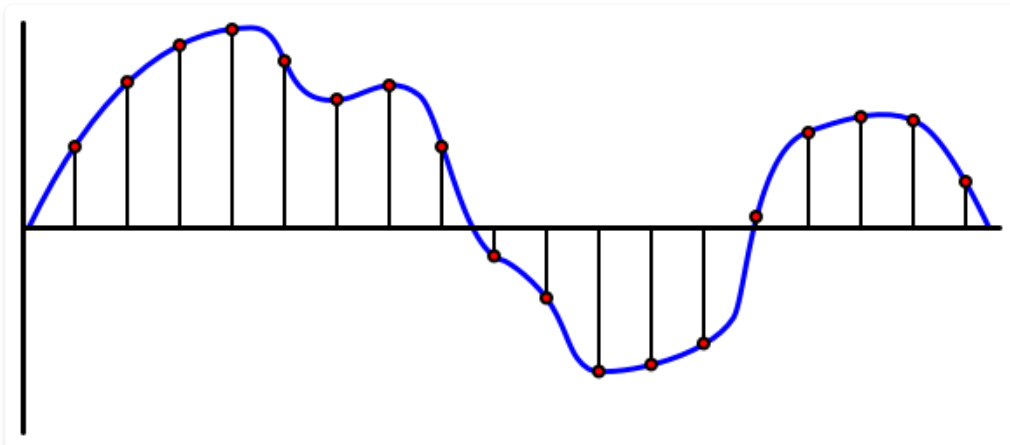



The ADC samples the incoming voltage and quantizes it to the nearest step in its resolution.

# Sampling: Snapshots of a Continuous World

The ADC can’t capture every instant, so it **samples**: reading the voltage at a fixed rate and keeping the dots, not the curve.

- **Sample rate** = readings per second
- Sample **too slowly** and you miss what happened in between
- Sample **faster** and you get a truer signal, at the cost of power, memory, and bandwidth



A few readings a minute is plenty for soil moisture. Audio needs thousands per second.

# Digital Sensors

A **digital** sensor sends its signal as 0s and 1s directly: no external ADC needed.

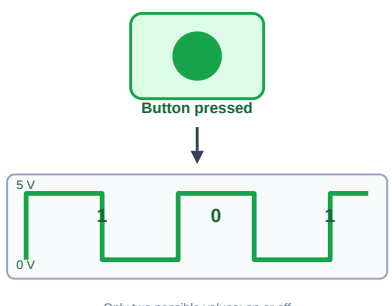
### 🔘 Simple

A button : 3.3 V = **1**, 0 V = **0**

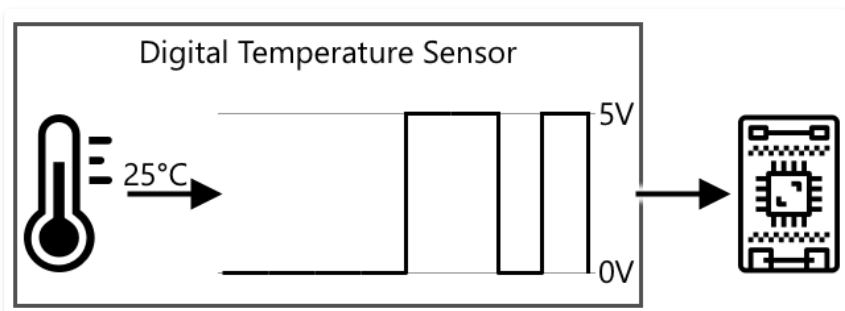
### 🌡️ Complex

Built-in ADC, sends **serialized** data

That range enables richer data too, like a temperature reading sent as a serialized value.




No in-between readings: the signal is always fully high or fully low.




The sensor already converts 25°C into a square-wave digital signal before it ever reaches the device.

Live preview, view online at the workshop site

2026-iot.abdeljabar.net

# Analog Inputs 🎚️

Read a potentiometer as a number from **0 to 4095** with the ESP32's ADC.

- Tell digital signals from analog ones
- Read a variable resistor with `analogRead()`
- Watch live readings on the Serial Monitor
- Convert a raw ADC count into a voltage

Key pin: potentiometer `4` (ADC)

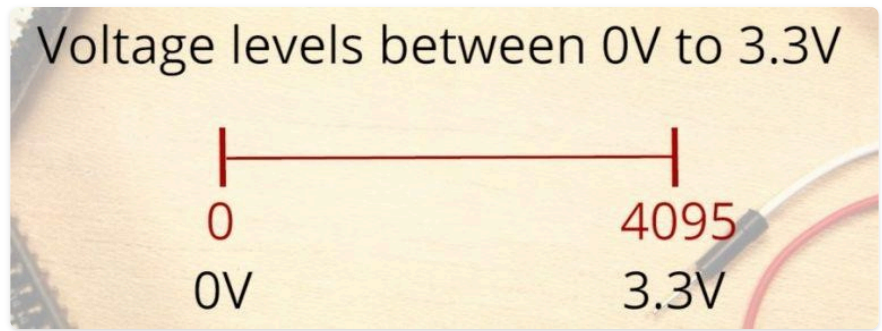
Open the guide ↗

# Analog vs Digital: the ADC

Project 1 saw only HIGH or LOW. An **analog** input measures a **continuous** voltage between 0 and 3.3 V and turns it into a number.

- The ESP32 ADC is **12-bit**, so readings run **0 to 4095**
- 0 V → **0**, 3.3 V → **4095**, everything in between is proportional
- Read it with `analogRead()`

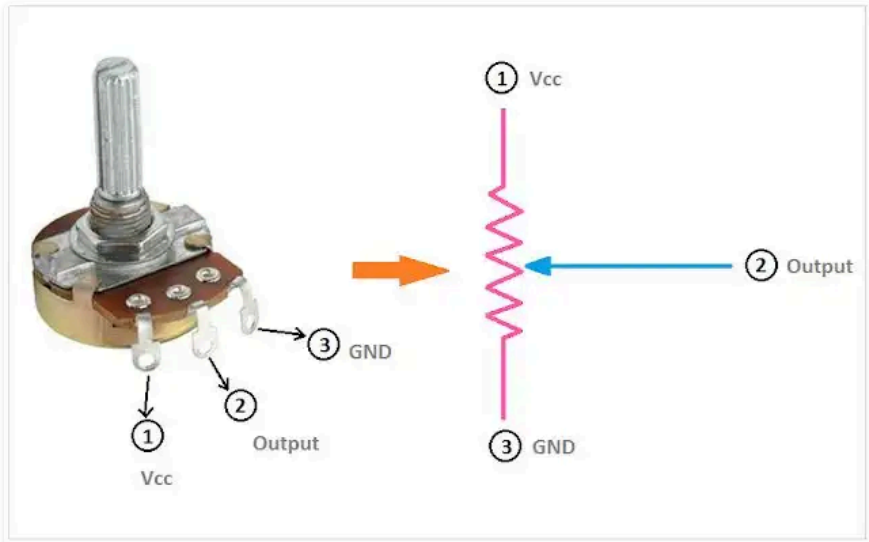
> ⚠ **Warning**
>
> We use **GPIO 4** (ADC2). ADC2 pins cannot be read while **Wi-Fi** is on, so in later Wi-Fi projects prefer an **ADC1** pin (32 to 39).



# What Is a Potentiometer?

A **potentiometer** is a **variable resistor**: three terminals, dividing voltage as the knob turns.

- Two outer legs connect across the supply: **Vcc** and **GND**, with a resistive track between them
- The middle leg, the **wiper**, slides along that track
- The wiper taps off whatever fraction of the voltage it's currently sitting at
- Turning the knob moves the wiper, changing the **output** voltage



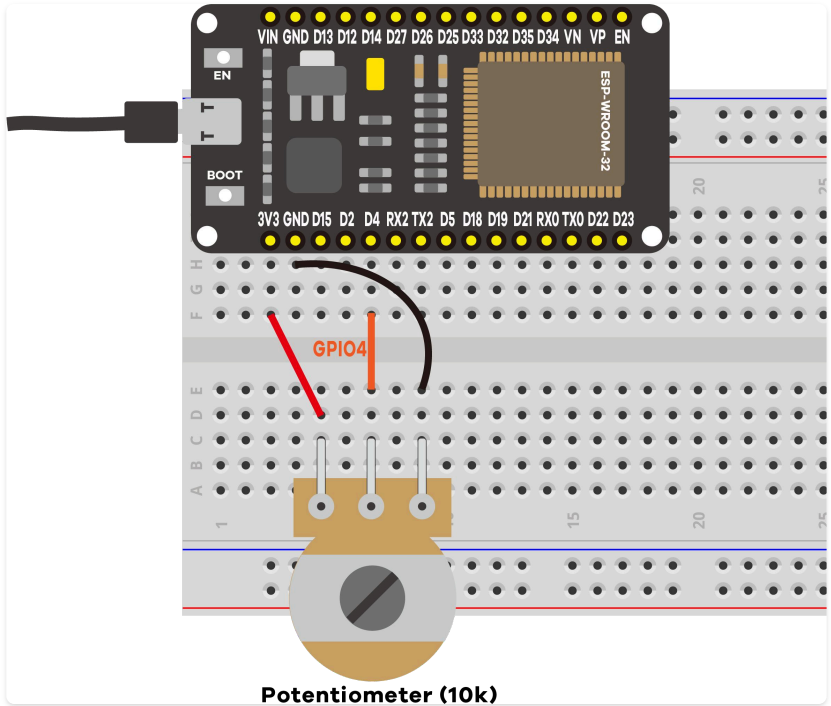

Source: onzuu.com

# Wiring the Potentiometer

A potentiometer is a **variable resistor** with three legs:

1. One outer leg → **3.3 V**
2. The other outer leg → **GND**
3. The middle leg (**wiper**) → **GPIO 4**

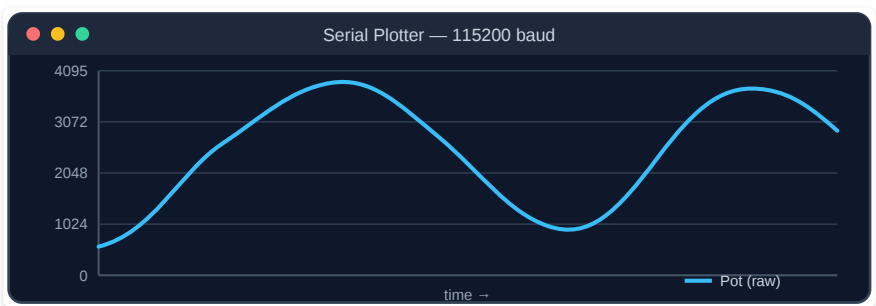
Turning the knob moves the wiper, tapping off anything from 0 V to 3.3 V, and the ADC reports the matching 0 to 4095 value. Any of the board's 15 ADC pins would work.



# Code: Read → Convert → Print

```ino
void loop() {
  potValue = analogRead(potPin);            // 1. READ: returns 0 .. 4095
  float voltage = potValue * 3.3 / 4095.0;  // 2. CONVERT to volts
  Serial.printf("Pot: %4d / 4095  (~%.2f V)\n", potValue, voltage);
  delay(500);                               // 3. slow down so it is readable
}
```

- `analogRead(pin)` returns a **0 to 4095** integer against the 3.3 V reference.
- `raw × 3.3 / 4095` converts to volts. 2048 is about **1.65 V**, half of 3.3V.
- `Tools → Serial Plotter` (115200) graphs the value live as you turn the knob.




The raw reading traced live as the knob turns, one point per loop().

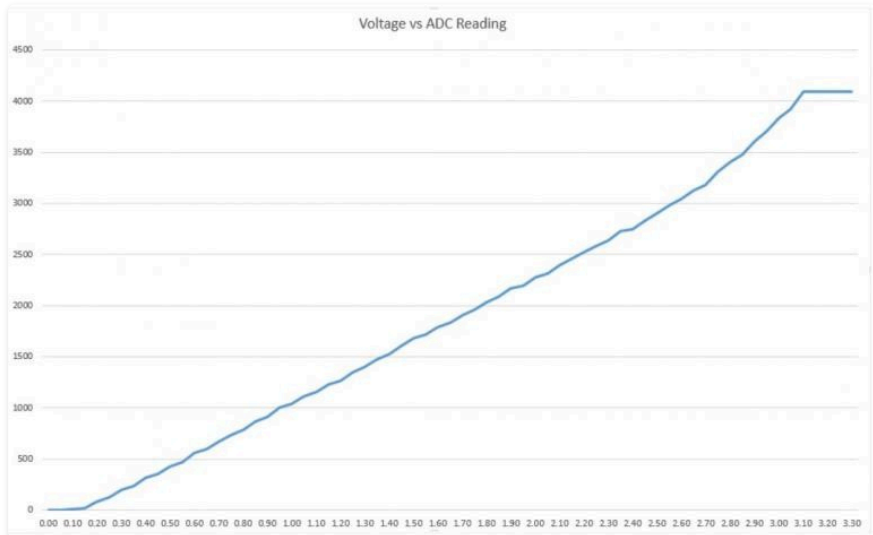
# The ADC Is Not Perfectly Linear

Ideally the value tracks the voltage in a straight line. In reality the readings **flatten near the ends**, so you cannot tell ~0 V from ~0.1 V, or ~3.2 V from ~3.3 V.

- Fine for a knob or a light sensor
- Matters for **precise** measurement

**What you learned:**

- The board has **15 ADC pins**, each 12-bit (0 to 4095)
- `analogRead()` is all it takes to read one
- The same GPIO 4 was a digital input in Project 1



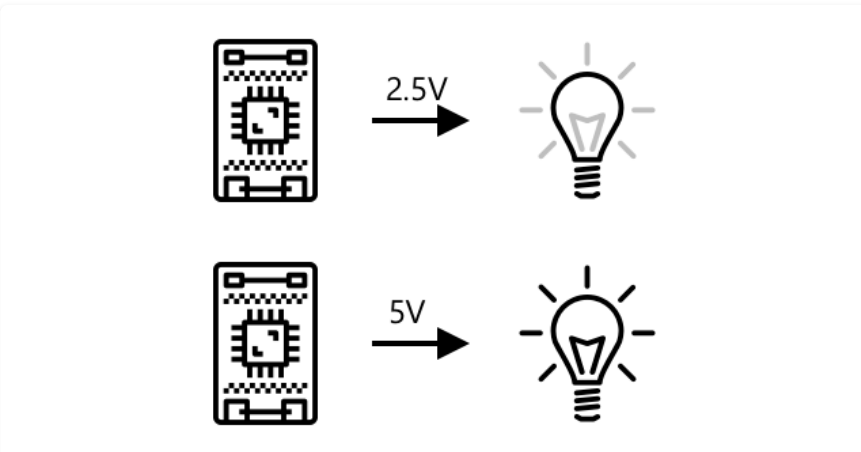
# Controlling Actuators 

Analog voltages, PWM, and simple on/off

# Analog Actuators & the DAC

An **analog** actuator responds to the **voltage** supplied, like a **dimmable light**.

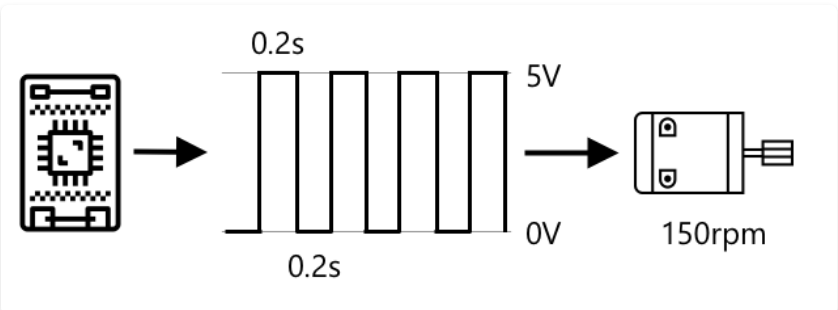
- The device is digital, so it needs a **DAC** (digital-to-analog converter)
- The DAC turns 0s and 1s into a real voltage the actuator can use




A lower voltage from the DAC dims the light; a higher voltage brightens it.

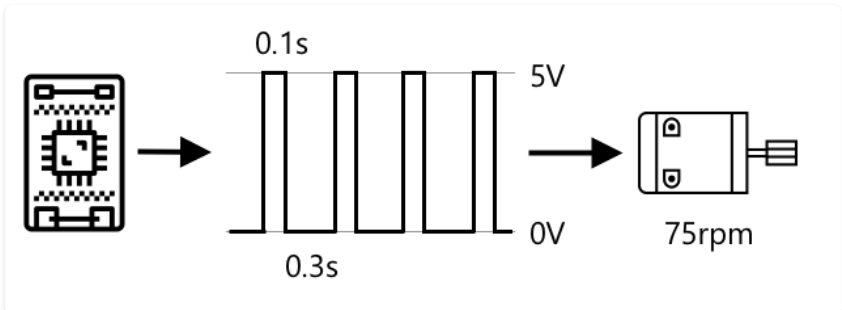
# PWM: Faking Analog with Digital

**Pulse-Width Modulation** sends rapid on/off pulses. The **duty cycle** (the % of time it's on) sets the effective power.




Wider on-pulses deliver more average power to the motor.

**50% duty** → motor at **150 RPM**




Narrower on-pulses mean less average power and a slower motor.

**25% duty** → motor at **75 RPM**

# Calculating the Duty Cycle

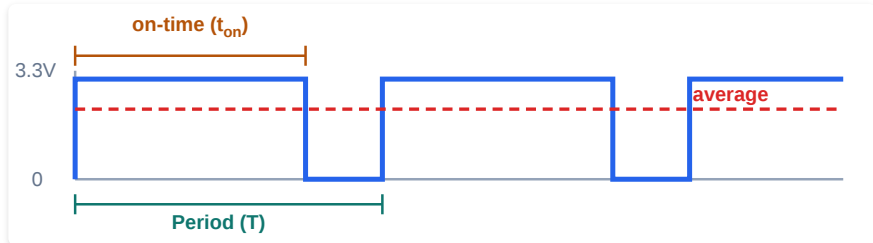
Each PWM cycle repeats over a fixed **period (T)**: the time for one on/off pulse.

- Duty cycle = $t_{on}$ / $T$, usually shown as a **%**

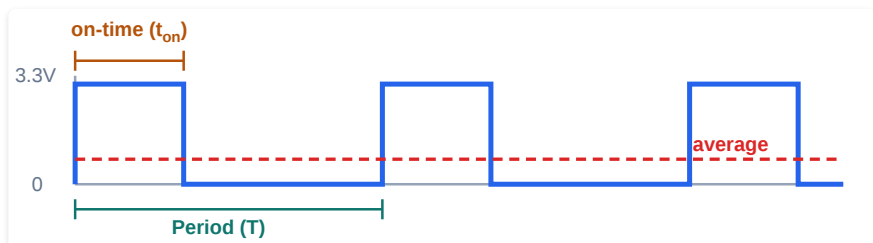
$$\text{Duty cycle} = \frac{t_{on}}{T} \times 100\%$$

- The average voltage seen by the actuator is just the duty cycle times the supply voltage

$$V_{avg} = \text{duty cycle} \times V_{supply}$$




**Duty cycle = 70%** · **average** = 0.70 × 3.3 V ≈ **2.31 V**



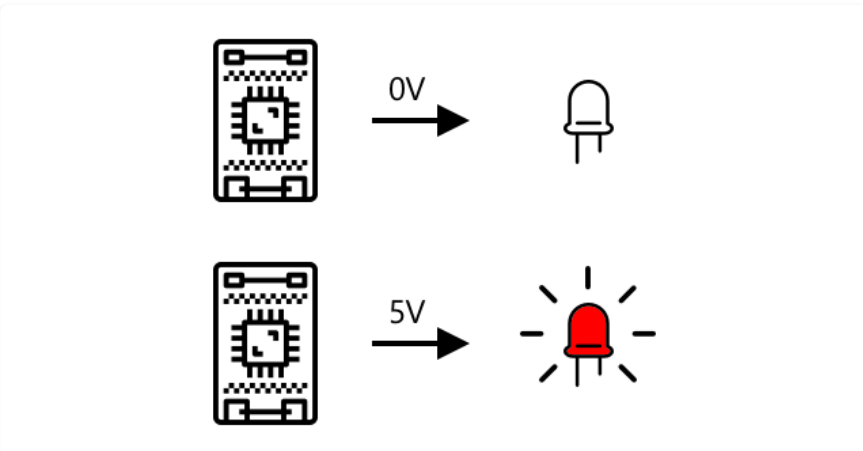

**Duty cycle = 25%** · **average** = 0.25 × 3.3 V ≈ **0.83 V**

# Digital Actuators

The simplest actuators have just **two states**: on or off.

- An **LED**: 0 V = off, 5 V = on
- A **relay**: a small voltage switches a larger circuit
- Complex ones (screens) need **libraries**

```
digitalWrite(LED_PIN, HIGH); // on
digitalWrite(LED_PIN, LOW);  // off
```




No in-between states: 0 V is fully off, 5 V is fully on.

Live preview, view online at the workshop site

2026-iot.abdeljabar.net

# PWM Analog Output 🌗

Fade an LED smoothly using the ESP32's built-in LED PWM controller.

- Understand pulse width modulation and duty cycle
- Use the 16-channel LED PWM controller, configurable frequency and resolution
- Drive a pin with `ledcAttach()` and `ledcWrite()` (Core v3.x+ API)
- Compare a digital on/off output with an analog-style dimming one

Key pin: LED `4` (PWM)

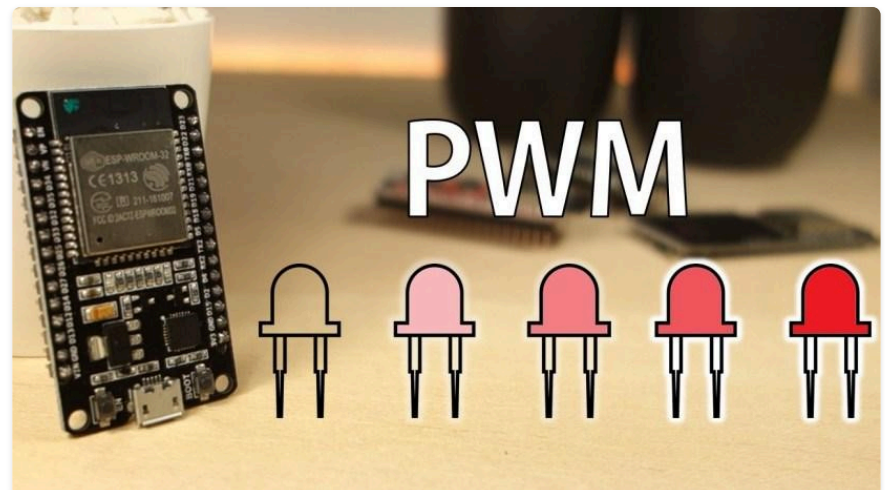
Open the guide ↗

# What Is PWM?

A digital pin is only ever fully **ON** (3.3 V) or **OFF** (0 V), it cannot output 1.6 V. But switch it on and off **very fast** and the LED's average brightness follows the **duty cycle**, the fraction of time it is ON.

- **0%** duty → dark
- **50%** duty → medium
- **100%** duty → full brightness

At **5000 Hz** the switching is far faster than your eye, so you see a steady brightness, not flicker. That is **Pulse Width Modulation**.



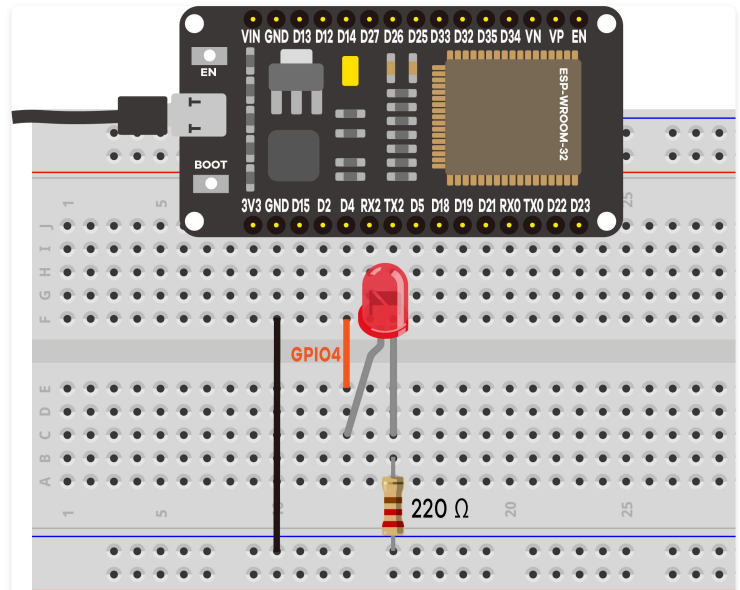
# Two Functions Run the Controller

The ESP32 has a dedicated **LED PWM controller**. With Core **v3.x+** it takes just two calls:

```
// setup(): enable PWM on the pin
ledcAttach(ledPin, 5000, 8); // pin, 5 kHz, 8-bit
// anytime: set the brightness
ledcWrite(ledPin, dutyCycle); // 0 = off .. 255 = full
```

- One call picks the pin, frequency, and resolution and allocates a channel for you
- **8-bit** resolution gives 256 levels (0 to 255)
- v3.x+ `ledcWrite()` takes the **pin**, not a channel

Wiring is three connections: GPIO 4 → **220 Ω** → LED anode, cathode → **GND**.

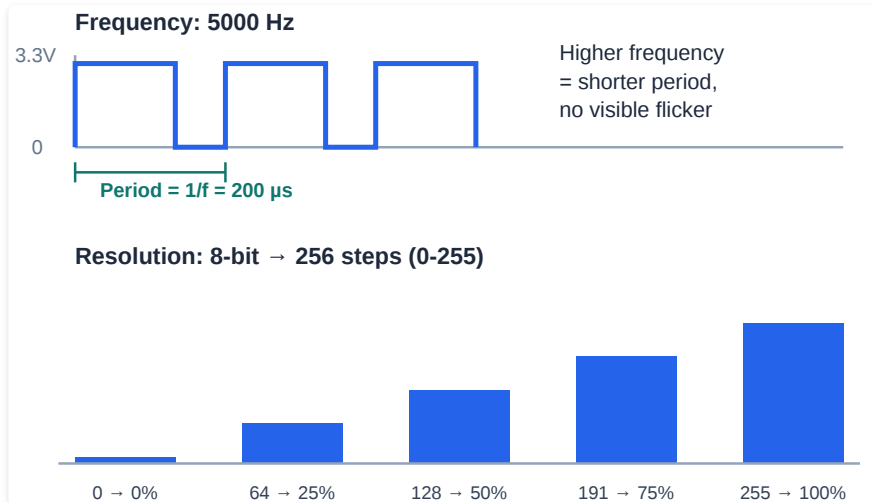


# Frequency, Resolution, Duty Cycle

`ledcAttach(pin, 5000, 8)` sets the two hardware settings; `dutyCycle` in `ledcWrite()` sets the brightness.

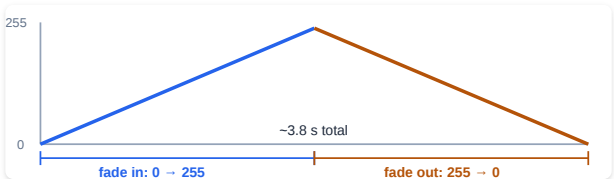
- **Frequency (5000 Hz)**: how often the pulse repeats. Period = 1 / frequency = **200 µs**
- **Resolution (8-bit)**: how many brightness steps fit in that period, **256** (0 to 255)
- **Duty cycle**: which step you picked, as a fraction of 255

> **Note**
>
> More resolution bits = smoother fades, at the cost of a slower maximum frequency.



# The Fade Loop

```
void loop() {
  for (int dutyCycle = 0; dutyCycle <= 255; dutyCycle++) {   // fade in
    ledcWrite(ledPin, dutyCycle);
    delay(15);
  }
  for (int dutyCycle = 255; dutyCycle >= 0; dutyCycle--) {   // fade out
    ledcWrite(ledPin, dutyCycle);
    delay(15);
  }
}
```



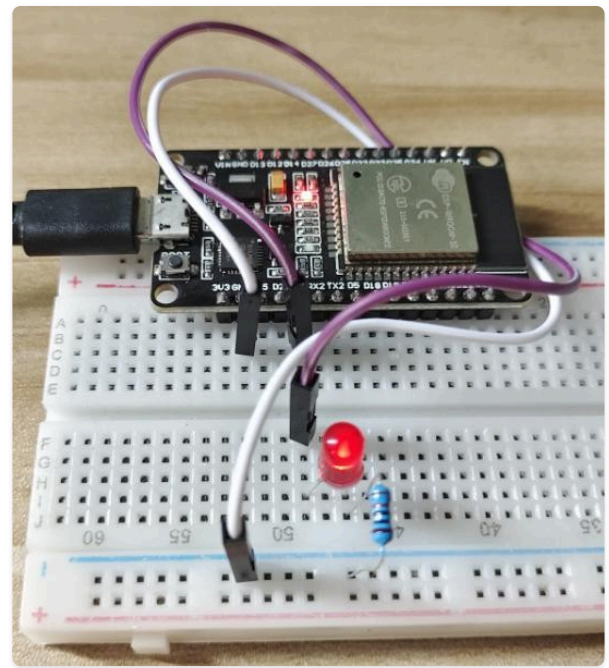

- Each step nudges the duty cycle by one; `delay(15)` spaces them, so a full fade is 256 × 15 ms ≈ **3.8 s**.
- The hardware does the fast switching; the code only says *how bright*.
- The sketch prints one line per **direction change**, not per step, so the monitor stays readable.

# It Breathes

The LED breathes smoothly brighter and dimmer, over and over.

**What you learned:**

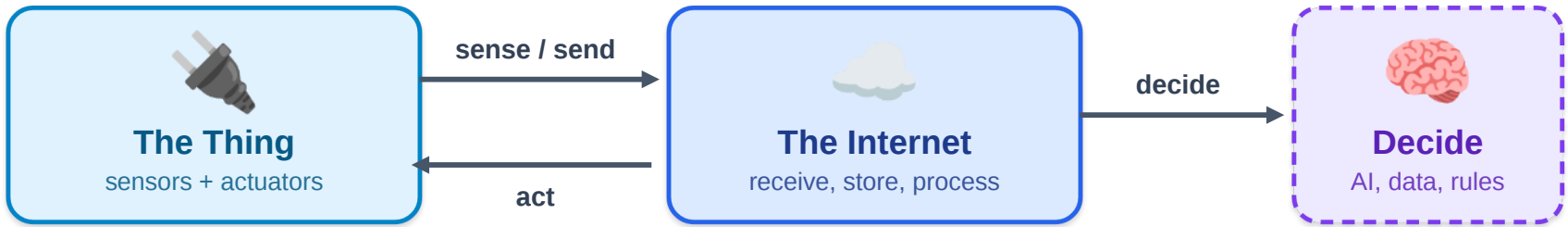
- Fast digital switching plus a duty cycle gives an **analog-looking** output
- Any output-capable pin can be a PWM pin
- Low PWM frequencies flicker; 5000 Hz is safe for LEDs
- The same trick sets motor speed and mixes RGB color (Project 6)



# Components of an IoT Application

The Thing, the Internet, and Decisions

# The Thing + The Internet




the loop: sense → send → decide → act

## 🔌 The Thing

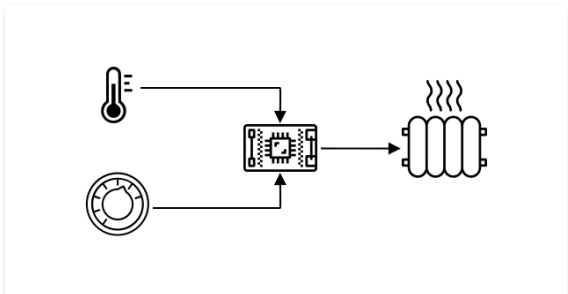
A device that interacts with the physical world, usually a small, low-power computer with **sensors** and **actuators**.

## ☁️ The Internet

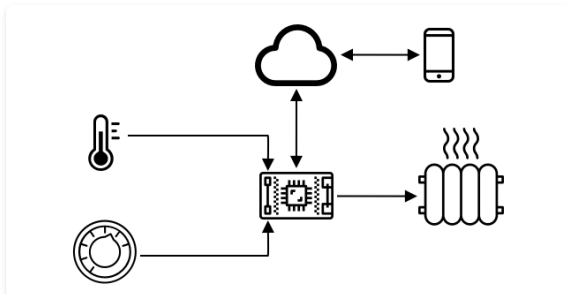
Cloud services the device connects to, in order to **receive** telemetry, **store & process** data, and **send commands** back.

> ⓘ **Note**
>
> Add **decision-making** (databases, AI, other data sources) and the loop is complete: **sense** → **send** → **decide** → **act**.

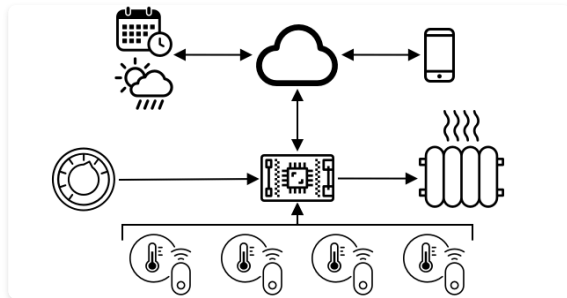
# A Thermostat Gets Smarter



**Basic**: temperature + dial → heater



**Connected**: cloud + phone app



**Smart**: AI + calendar + weather

Each step adds more data and more decision-making in the cloud.

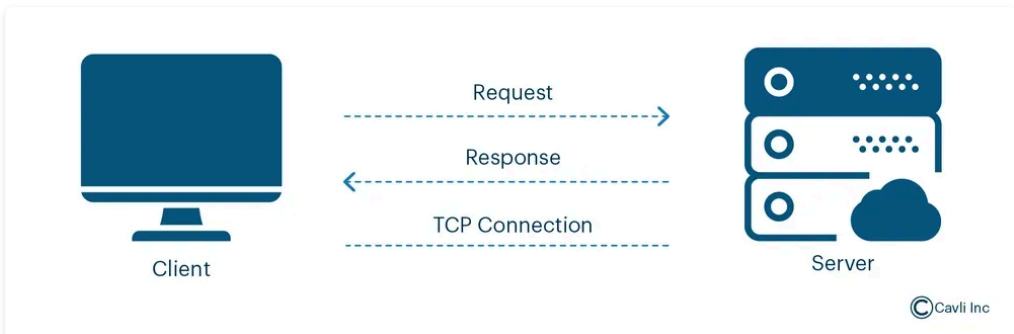
# IoT Communication 

How devices and the cloud talk

# Request / Response

HTTP is a **client-server** protocol: the device (or browser) always initiates.

- The client sends a request (a method + a URL)
- The server processes it and sends back a response
- Common methods: GET (read), POST (create), PUT (update), DELETE (remove)
- HTTP is stateless: each request is handled with no memory of previous ones

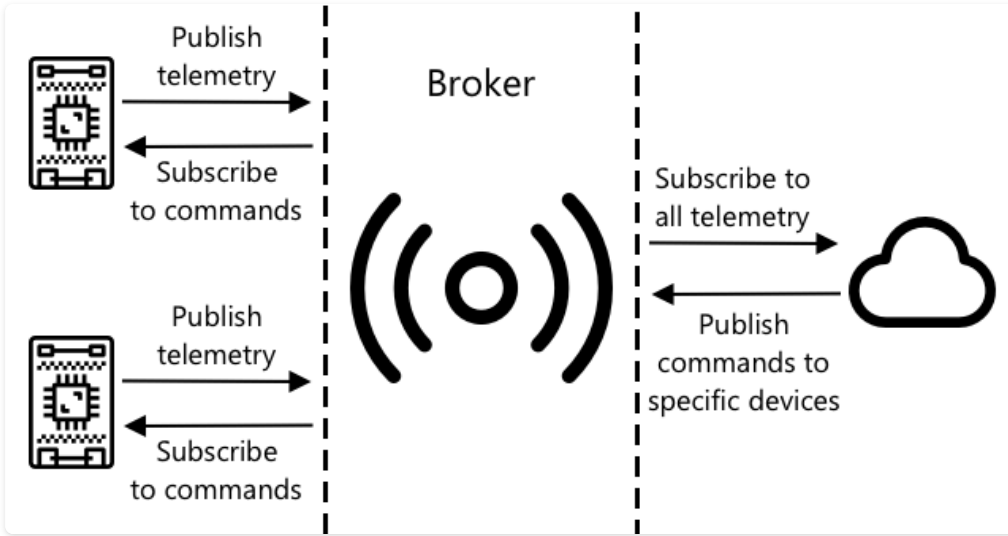



Source: Cavli Wireless

# Publish / Subscribe

Most IoT messaging uses pub/sub through a broker:

- Devices publish telemetry to topics
- Devices subscribe to command topics
- The broker routes messages between them
- Cloud services subscribe to telemetry & publish commands




Publishers and subscribers never talk directly; the broker routes every message between them.

# Popular Protocols

Devices and the cloud exchange messages using one of a few common protocols:

## MQTT

Pub / Sub

Lightweight, designed for IoT. *(Our focus.)*

⚡ **Lightweight**

## AMQP

Exchange + Queue

Robust enterprise messaging.

⚖️ **Heavier, guaranteed delivery**

## HTTP / HTTPS

Request / Response

The web standard: simple.

📶 **Higher overhead**

**Important**

**MQTT** is the most popular protocol for IoT: small, efficient, and built for constrained devices.

# Try It on Your Own Laptop First

Before the ESP32 does this in C++, run the same request/response loop on your laptop, no code needed.

- Make a page to serve, one line:

```
echo "<h1>Hello from my laptop!</h1>" > index.html
```

- Then start the server in that same folder:

```
python3 -m http.server 8000
```

- Find your laptop's IP address ( `ipconfig` on Windows, `ifconfig` or `ip addr` on Mac/Linux)
- On your **phone**, connected to the **same Wi-Fi**, open `http://<your-ip>:8000`
- Your phone's browser is now the client, your laptop is the server

**Tip**

Python ships this server built in: no install, no dependencies. It serves `index.html` from the folder you ran it in.

**Note**

Same request, response, client, server story as the last slide, just running on your own machine instead of the ESP32. That's exactly what Project 5 builds next, in C++.

Live preview, view online at the workshop site

2026-iot.abdeljabar.net

# LED Switch Web 🌐

Toggle two LEDs from a web page hosted directly on the board, no cloud involved.

- Connect to Wi-Fi and find the board’s IP address
- Host a web page from the ESP32 itself
- Read an HTTP request and route it to an action
- Keep the page in sync with each output’s state

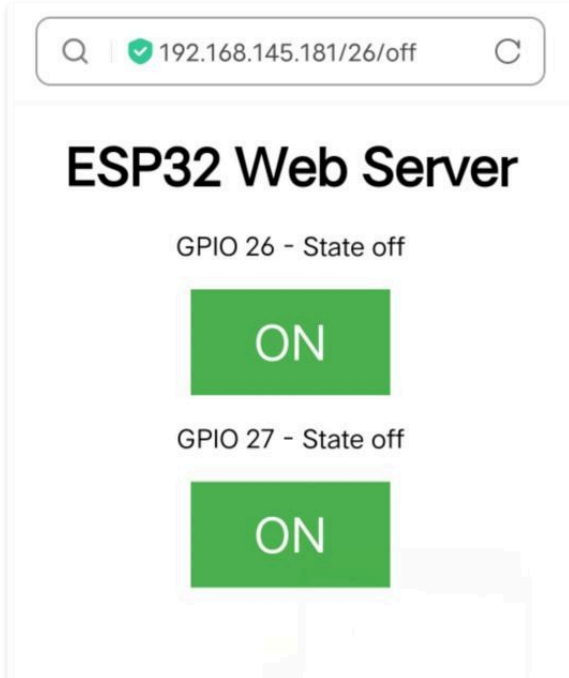
Key pins: LEDs `26`, `27`

Open the guide ↗

# How an ESP32 Web Server Works

1. The ESP32 joins your Wi-Fi and gets an **IP address**
2. It **listens** on port 80 for browsers
3. Visit `http://<ESP_IP>/` and it sends back an HTML page
4. Each button is a link to a URL like `/26/on`; the board sees that path, switches the LED, and re-sends the page with the new state

That request-then-respond loop is the heart of **every** web-server project in this kit (5 to 9).

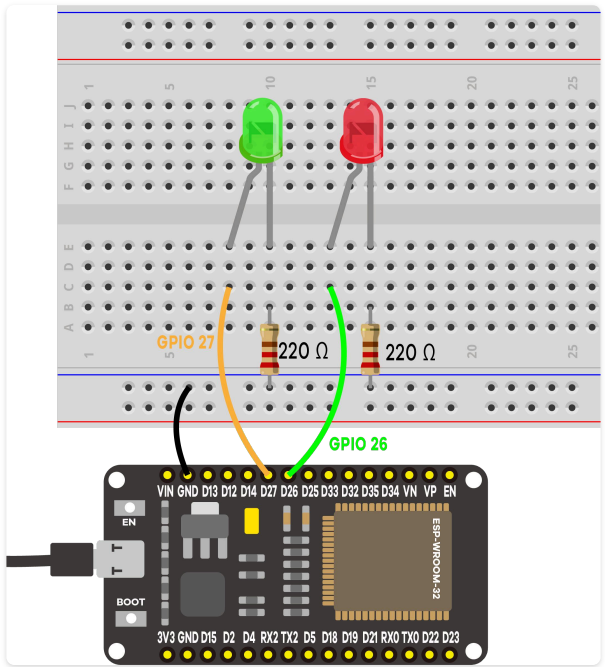


# Connecting to Wi-Fi

Set your network name and password at the top of the sketch, then it joins and prints a clickable address:

```
WiFi.begin(ssid, password);   // returns at once
while (WiFi.status() != WL_CONNECTED) {
  delay(500); Serial.print("."); // a dot per 0.5 s
}
Serial.println(WiFi.localIP()); // type this in a browser
server.begin();                 // listen on port 80
```

The ESP32 radio is **2.4 GHz only**. Endless dots mean wrong credentials or a 5 GHz network. Wiring: two LEDs, each GPIO → 220 Ω → anode, cathode → GND.



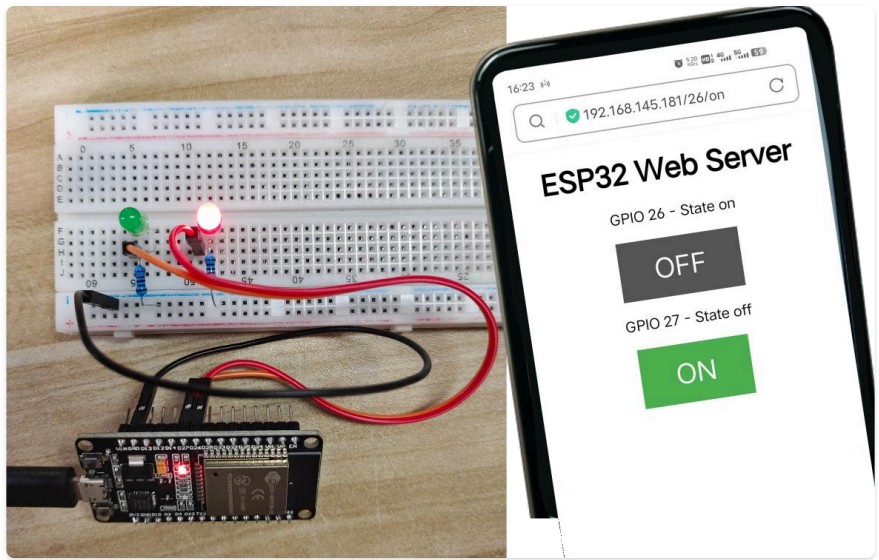
# HTTP Is Just Text

```
WiFiClient client = server.available();   // did a browser connect?
// ... read bytes into `header` until the BLANK line that ends the request ...
client.println("HTTP/1.1 200 OK");        // status line: it worked
client.println("Content-type:text/html"); // render the reply as HTML
client.println();                         // blank line ends our header
if (header.indexOf("GET /26/on") >= 0) {  // the URL carries the command
  digitalWrite(output26, HIGH); output26State = "on";
}
client.println("<!DOCTYPE html>...");     // build the page, with current state
```

- A request is text lines ended by a **blank line**; a response is a status line, headers, a blank line, then the page.
- The **URL carries the command**: links like `/26/on`, and `indexOf()` returns `>= 0` when the browser asked for one.
- Port **80** is the default for `http://`, so the typed address needs no `:port`.

# The Board Tracks Its Own State

The ESP32 cannot read back what an output pin is doing, so it keeps `output26State` and `output27State` and writes the current value into every page. The browser always shows what the board **believes**.




**What you learned:**

- Your board is now a **server**: the step that makes it IoT
- It joins 2.4 GHz Wi-Fi and the router hands it an IP
- HTTP is text, and the URL carries the command
- Tracked state matters when a button changes it too (Project 8)

# Inside a Microcontroller 

CPU · Memory · I/O

# 🧠 The CPU

The **brain** that runs your code, driven by a **clock** that ticks millions or billions of times a second.

### 🔁 Each Tick

One step of the **fetch-decode-execute** cycle

### 📏 Speed Units

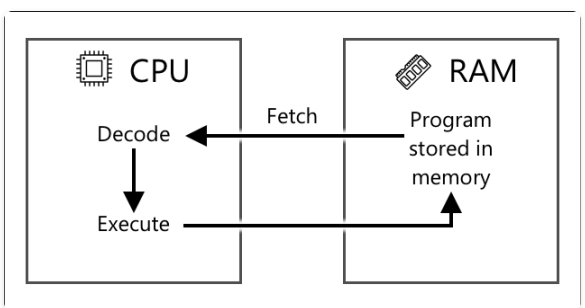
1 MHz = 1M ticks/s · 1 GHz = 1B ticks/s

### ⚙️ MCU vs PC

MCU: ~**120 MHz** · PC: several **GHz**

### 🔋 Power Tradeoff

Slower = cooler & lower power → months on a battery




Every clock tick advances the CPU through this same three-step cycle.

# 💾 Memory

Two kinds, both **tiny** next to a PC:

### 💾 Program Memory

Non-volatile: keeps code without power

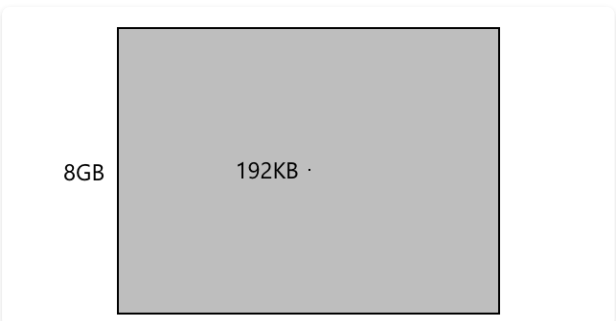
MCU: **4 MB** · PC: **500 GB**

### ⚡ RAM

Volatile: holds running data, lost on power-off

MCU: **192 KB** · PC: **8 GB**

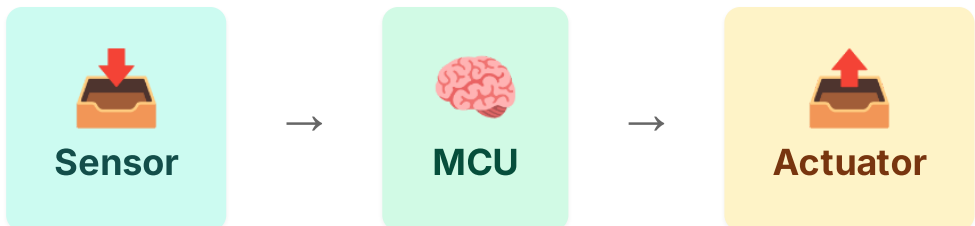
A PC has roughly **40,000×** more RAM.




The dot in the center is 192 KB of RAM vs 8 GB of RAM

# 🔌 Input / Output

Microcontrollers talk to the world through **general-purpose I/O (GPIO)** pins, each configurable in software:




Same GPIO pins, configured as input or output in software

**🧠⬅️ Input**

Read values **from sensors** with `digitalRead()` / `analogRead()`

**🧠➡️ Output**

Send signals **to actuators** with `digitalWrite()` / `analogWrite()`

Live preview, view online at the workshop site

2026-iot.abdeljabar.net

# Output State Sync 🔄

One LED, controlled by both a web toggle and a physical button, kept in sync.

- Control one output from two independent sources without them fighting
- Fix a stale web page by polling the server
- Use `setInterval()` and `XMLHttpRequest` to refresh state with no page reload
- Debounce a button with `millis()` while the server keeps responding

Key pins: LED `2`, button `4`

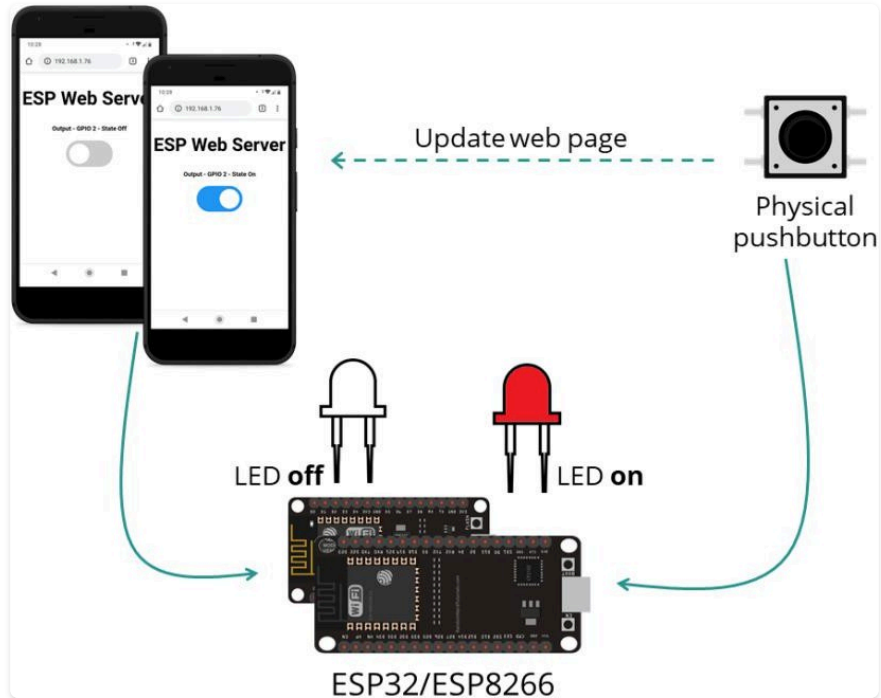
Open the guide ↗

# One Output, Two Controls

Control one LED (GPIO 2) from **two places at once**: a web-page toggle and a physical button, and keep the page in sync either way.

The trick is a **single source of truth**:

- Both the web route and the button change one variable, `ledState`
- Neither writes the pin directly; `loop()` applies `ledState`
- Agreeing on a variable is what stops the two controls fighting



# A Web Page Goes Stale, So Poll It

The page cannot know the button was pressed, so it **asks**. Every second the browser fetches `/state` and redraws its toggle, with no reload:

```
setInterval(function () {              // every 1000 ms
  xhttp.open("GET", "/state", true);  // ask the ESP32 for the real state
  xhttp.send();                       // reply is just "0" or "1"
}, 1000);
```

- **Web toggle** → `/update?state=…` → LED changes. **Button** → LED changes in `loop()`. **Either way**, the 1-second poll refreshes the page.
- The `/state` route is tiny: `request->send(200, "text/plain", String(digitalRead(output)))` reads the **actual pin**, not the variable.
- This "poll the server" pattern is how simple dashboards stay live.

# Debounce Without Blocking

The button is debounced with `millis()`, not `delay()`, so the server keeps responding while it is read:

```
int reading = digitalRead(buttonPin);
if (reading != lastButtonState)
  lastDebounceTime = millis();                    // input moved: restart the timer
if (millis() - lastDebounceTime > debounceDelay) {// stable for 100 ms?
  if (reading != buttonState) {
    buttonState = reading;
    if (buttonState == LOW) ledState = !ledState; // act on one edge
  }
}
digitalWrite(output, ledState);                   // apply the agreed state each pass
```

- Project 1's blunt `delay(50)` would stall the web server; the `millis()` debounce does not.
- Project 7's empty `loop()` is exactly the room this project needed.

# Sync in Action

Toggle from the web and the LED changes. Press the button and the LED changes **and** the web toggle flips within a second.

**What you learned:**

- Route controls through **one state variable**; two sources of truth give two answers
- A web page is a **snapshot**; polling keeps it live
- Polling is wasteful (a request per second, per browser), which is why Project 11 moves to **MQTT**
- Non-blocking debounce matters once the program has another job

# Thank you!

For spending the day building the Internet of Things with us.

Questions? Ask now, or write to me any time.

Slides, labs, and resources

KGSP Summer Enrichment Program 2026
KAUST Academy, KAUST

Salah Abdeljabar
salah.abdeljabar@kaust.edu.sa